

Pearls AQI Predictor

A Serverless End-to-End Machine Learning System for
3-Day Air Quality Index Forecasting

Technical Project Report

Prepared by: Muhammad Mahad
Date: September 2026

Repository: <https://github.com/MuhammadMahad461/aqi-predictor>

Live App: <https://aqi-predictor-jwg3mdpy7exxmnmncvfjyk.streamlit.app/>

Table of Contents

Table of Contents	2
1. Executive Summary.....	3
2. Project Objectives	3
3. System Architecture	3
3.1 Component Breakdown	3
4. Data Sources.....	4
5. Feature Engineering	4
6. Historical Backfill.....	4
7. Feature Store & Model Registry	5
8. Model Training & Evaluation.....	5
9. Automation (CI/CD).....	5
10. Dashboard	6
11. Engineering Challenges & Solutions.....	6
12. Results & Current System Status	7
13. Limitations & Future Work.....	7
14. Conclusion.....	7

1. Executive Summary

This report documents the design, implementation, and deployment of the Pearls AQI Predictor — a fully serverless machine learning system that forecasts the Air Quality Index (AQI) for a target city three days in advance. The project was built end-to-end across eight stages: feature engineering, historical backfilling, feature store integration, model training and evaluation, pipeline automation, dashboard deployment, and analytical reporting.

The completed system ingests live weather and pollution data hourly, retrains forecasting models daily without manual intervention, and serves live predictions through a public web dashboard with built-in hazard alerting. The project satisfies the core deliverables of an end-to-end AQI prediction system, a scalable automated pipeline, an interactive dashboard, and this documenting report.

2. Project Objectives

The goal of the project, as specified in the original brief, was to build a 100% serverless AQI prediction service composed of the following components:

- A feature pipeline that fetches raw weather and pollutant data and computes derived features.
- A feature store and model registry for centralized, versioned storage of features and trained models.
- A training pipeline that experiments with multiple ML models and evaluates them using standard regression metrics.
- An automated CI/CD system that keeps data and models continuously up to date.
- A web application dashboard that surfaces live and forecasted AQI values to end users.

3. System Architecture

The system follows a modular, event-driven pipeline architecture with four independently deployable components, connected through a managed feature store and model registry (Hopsworks). The high-level data flow is as follows:

Weather & Pollution APIs → Feature Pipeline → Hopsworks Feature Store → Training Pipeline → Hopsworks Model Registry → Streamlit Dashboard

Each stage runs independently and communicates only through the feature store / model registry, which decouples data collection from model training and from serving — a standard MLOps pattern that allows any one component to be modified, retried, or scaled without affecting the others.

3.1 Component Breakdown

Component	Responsibility	Technology
Feature Pipeline	Fetches live weather/pollution data, engineers features, writes to feature store	Python, OpenWeather API, Open-Meteo
Historical Backfill	Constructs a multi-week training dataset from historical sources	Python, pandas

Component	Responsibility	Technology
Feature Store	Central, versioned storage for engineered features	Hopsworks (HUDI format)
Training Pipeline	Trains, evaluates, and registers the best-performing model	Scikit-learn, Hopsworks Model Registry
Automation	Schedules pipeline execution without manual triggering	GitHub Actions
Dashboard	Serves live current AQI, 3-day forecast, trend, and alerts	Streamlit, Plotly

4. Data Sources

Two complementary, free-tier data sources were used to maximize historical coverage while keeping the system entirely serverless and cost-free:

- OpenWeather Air Pollution API — live and historical PM2.5, PM10, NO₂, O₃, CO, SO₂, and NH₃ concentrations.
- Open-Meteo Historical Weather API — historical temperature, humidity, surface pressure, and wind speed, requiring no API key.

The US Air Quality Index was computed directly from PM2.5 concentration using the standard EPA piecewise-linear breakpoint formula, rather than relying on OpenWeather's coarser 1–5 category scale. This gives the model a continuous, well-understood numeric target and makes the derived AQI values directly interpretable and comparable to official air quality reporting.

5. Feature Engineering

The feature pipeline computes the following inputs for each hourly observation:

- Time-based features: hour of day, day, month, and day of week — capturing diurnal and seasonal pollution patterns.
- Weather features: temperature, humidity, atmospheric pressure, and wind speed.
- Pollutant concentrations: PM2.5, PM10, NO₂, O₃, CO, SO₂, and NH₃.
- Derived features: current computed AQI, and AQI change rate (difference from the previous hourly reading).

The prediction target, `target_aqi_3d`, is constructed by shifting the computed AQI series forward 72 hours, so that each row's feature set is paired with the AQI value three days later. Rows without a valid future value (the most recent 72 hours of data) are excluded from training.

6. Historical Backfill

To bootstrap a usable training dataset without waiting weeks for hourly collection to accumulate, approximately 60 days of historical weather and pollution data were backfilled in a single batch. Pollution history was pulled from OpenWeather's `air_pollution/history` endpoint; weather history was pulled from Open-Meteo's archive API. The two series were merged on nearest timestamp using pandas' `merge_asof`, then passed through the same feature engineering and target-construction logic used by the live pipeline, ensuring consistency between backfilled and live-collected data.

7. Feature Store & Model Registry

Hopsworks was used as the managed feature store and model registry, providing:

- A queryable, versioned Feature Group (aqi_features) storing all engineered features with timestamp as the event-time column.
- A Model Registry entry (aqi_forecast_model) storing the serialized best-performing model alongside its evaluation metrics and an input schema example.

The feature group was configured with the HUDI time-travel format rather than the default Delta format, after Delta's underlying HDFS/Kerberos writer proved incompatible with the Windows development environment used for this project — a practical engineering decision documented further in Section 11.

8. Model Training & Evaluation

The training pipeline pulls the full feature history from Hopsworks, applies a chronological (non-random) train/test split — appropriate for time-series data, since evaluating on randomly shuffled data would leak future information into training — and trains two candidate models:

- Ridge Regression — a regularized linear baseline, fast to train and resistant to overfitting on correlated pollutant features.
- Random Forest Regressor — a non-linear ensemble model capable of capturing interaction effects between weather and pollutant variables.

Each model is evaluated using three standard regression metrics:

- RMSE (Root Mean Squared Error) — penalizes large forecasting errors most heavily.
- MAE (Mean Absolute Error) — average magnitude of forecast error, in AQI points.
- R^2 (Coefficient of Determination) — proportion of variance in future AQI explained by the model.

The model with the lowest test-set RMSE is automatically selected and registered in the Model Registry, along with its metrics, so that the dashboard always serves the best available model without manual selection.

Result summary: insert your actual RMSE / MAE / R^2 values from the training run output here for both Ridge Regression and Random Forest, and state which model was selected as best.

9. Automation (CI/CD)

Two GitHub Actions workflows remove the need for manual pipeline execution:

Workflow	Schedule	Purpose
feature_pipeline.yml	Every hour (cron)	Fetch latest weather/pollution data and append to the feature store
training_pipeline.yml	Once daily (cron)	Retrain and re-evaluate models, update the registry with the best performer

Both workflows run on GitHub-hosted Ubuntu runners, install dependencies from requirements.txt, and authenticate to Hopsworks and OpenWeather using repository secrets rather than committed

credentials. Each workflow can also be triggered manually via `workflow_dispatch` for on-demand testing.

10. Dashboard

The dashboard is built with Streamlit and deployed publicly on Streamlit Community Cloud. It presents:

- The current AQI value with a color-coded category badge (Good through Hazardous, following EPA conventions).
- An automatic hazard warning banner when air quality is unhealthy or worse.
- The model's 3-day-ahead AQI forecast, shown alongside the change from the current reading.
- A 72-hour historical AQI trend chart, rendered with Plotly.

The dashboard loads the latest registered model and the most recent feature row directly from Hopsworks at runtime, so it always reflects the most recently trained model and most recently collected data, with no redeployment required when the underlying model or data changes.

11. Engineering Challenges & Solutions

Several non-trivial engineering obstacles were encountered and resolved during development, summarized below as they reflect meaningful problem-solving beyond the initial pipeline design:

Challenge	Root Cause	Resolution
Hopsworks certificate storage failure on Windows	Client hardcodes a Unix-style /tmp path	Manually created the equivalent C:\tmp directory
Feature group insert failure (Delta/HDFS error)	Delta writer depends on an HDFS/Kerberos library unsupported on Windows	Switched feature group storage format to HUDI
GitHub Actions dependency install failures	requirements.txt generated via pip freeze on Windows included invalid/unavailable package versions (e.g. an unpublished scipy version) and platform-specific artifacts	Rebuilt requirements.txt from a minimal, loosely-pinned dependency list and re-froze it from a clean resolve
Streamlit Cloud deployment failure (protobuf conflict)	hopsworks required protobuf<5.0.0 while the pinned streamlit version required protobuf>=5.26.1 — mutually exclusive constraints	Downgraded the streamlit version range to one compatible with hopsworks' protobuf constraint
Git push rejected / merge conflicts in requirements.txt	Remote had diverging commits; requirements.txt was saved with an incompatible text encoding under PowerShell redirection	Pulled and resolved conflicts manually; standardized the file on UTF-8 encoding

These issues, while resolved individually, point to a broader lesson applied throughout the project: environment-specific dependency pinning (via a raw pip freeze on a single Windows development machine) is fragile across deployment targets. The eventual fix — deriving a minimal explicit dependency list and letting the package resolver compute compatible versions — is the more robust long-term practice and was adopted for the final requirements.txt.

12. Results & Current System Status

At the time of writing, the system is fully deployed and operating autonomously:

- The feature pipeline has been verified to run successfully on an hourly GitHub Actions schedule.
- The training pipeline has been verified to run successfully on a daily GitHub Actions schedule and registers an updated model in Hopsworks.
- The Streamlit dashboard is live and publicly accessible, serving current AQI, hazard alerts, and 3-day forecasts sourced live from the feature store and model registry.

13. Limitations & Future Work

- Historical training data is currently limited to roughly 60 days; extending the backfill window would likely improve model accuracy, particularly for capturing seasonal pollution patterns.
- Only Ridge Regression and Random Forest were evaluated; deep learning approaches (e.g. LSTM sequence models) were identified as a future extension but not implemented in this iteration.
- SHAP-based feature importance explanations and formal exploratory data analysis were scoped in the original requirements but were deprioritized in this iteration in favor of completing and stabilizing the end-to-end deployed system; both remain natural next steps given the pipeline infrastructure already in place.
- The system currently supports a single city; generalizing the feature and training pipelines to accept a city parameter would allow multi-city deployment with minimal architectural change.
- Hazard alerting is currently limited to in-dashboard banners; push or email notifications were considered but not implemented.

14. Conclusion

This project delivered a working, autonomous, serverless AQI forecasting system spanning data ingestion, feature engineering, model training, automated retraining, and public deployment. Beyond satisfying the functional requirements of the original brief, the project involved substantial real-world engineering work in diagnosing and resolving cross-platform dependency, storage-format, and deployment-environment conflicts — the kind of operational friction that characterizes production MLOps work as much as the modeling itself. The result is a system that continues to collect data and retrain models without further manual intervention, and that serves live, explainable predictions to end users through a public dashboard.